AI 데이터 전처리 실습

Pandas 기초

© KOPO-Dept.
AI Engineering Limited

학습 목차

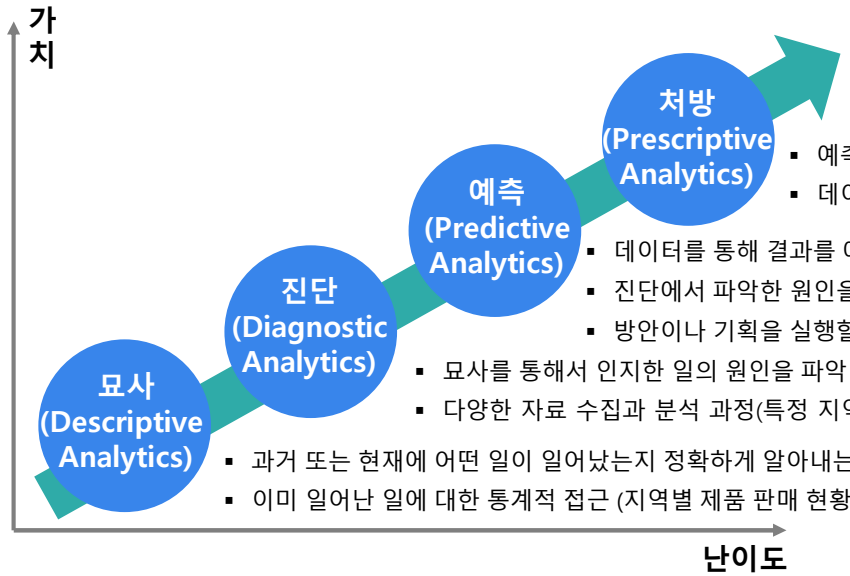
- 다양한 예제를 통한 데이터 분석 및 시각화에 앞써,
  - 1. 판다스 입문(기초)
  - 2. 데이터 입출력 (CSV, Excel, Json, etc.)
  - 3. 데이터 분석(전처리, 통계분석 등)/시각화
- 위 3가지에 대해서 기초 수업 진행 예정

참고도서
- **파이썬 머신러닝 판다스 데이터 분석**
- **파이썬으로 데이터 주무르기**
- **데이터 분석을 위한 전처리와 시각화 with 파이썬**
- **데이터로 전문가처럼 말하기**

데이터 분석

- 각종 도구, 기술, 프로세스를 활용하여 **원시 데이터로부터 유의미한 내용(인사이트 등)를 찾는 것**
  - 데이터 간의 관계 파악
  - 파악된 데이터 관계를 기반으로 원하는 결과를 만들어 내는 과정
    - 데이터: 숫자, 문자, 이미지, 오디오, 동영상 등의 형식으로 나타낸 모든 것을 의미

  - 분석 가치 단계 (Gartner, Analytic Value Escalator)



- 과거 또는 현재에 어떤 일이 일어났는지 정확하게 알아내는 데이터 분석
- 이미 일어난 일에 대한 통계적 접근 (지역별 제품 판매 현황, 고객 만족도 등)

- 묘사를 통해서 인지한 일의 원인을 파악 · 이해하는 데이터 처리
- 다양한 자료 수집과 분석 과정(특정 지역/계절에서 제품 판매가 높은/낮은 원인 파악)

- 데이터를 통해 결과를 예측하는 데이터 분석
- 진단에서 파악한 원인을 활용하여 기존 문제를 해결하는 방안 마련, 새로운 제품이나 서비스 기획
- 방안이나 기획을 실행할 경우의 성과 · 발생할 문제 분석(개선된 제품의 판매량 예측, 서비스 만족도 예측 등)

- 예측을 바탕으로 원하는 결과를 달성하기 위한 최적의 과정(최적화)을 수립하고 실행
- 데이터 분석을 중요하게 여기는 가장 큰 원인 (정부 예산 비율 설정, 투자 방안 설계, 투자 상품 출시 등)

*출처: https://www.gartner.com/en/conferences/emea/data-analytics-uk

데이터 분석 활용 사례

- 국외 활용사례
  - 구글, 아마존, DHL, 넷플릭스, 스타벅스 등 국외 기업들이 데이터 분석 기술을 활용

분류	국외 기업	활용 내용
고객 관계 관리 / 경험의 변화	아비바생명	고객 맞춤형 보험 상품 제공
고객 관계 관리 / 경험의 변화	사우스웨스트항공	고객 맞춤형 광고
	타겟	고객 맞춤형 상품 프로모션
	넷플릭스	시청자들의 행동 관찰, 선호도 분석을 통한 맞춤형 콘텐츠 제작
	스타벅스	데이터 기반의 상권 분석, 소비자 정보 기반의 추천 서비스
내부 프로세스 / 효율성 개선	DHL	물류 효율화, 투자 의사결정에 참고
	자라(Zara)	효율적인 물류 배송망 운영
	구글	데이터 센터 성능 및 에너지 사용 최적화
	테슬라	데이터 분석을 통해 의사결정을 내리는 데이터 드리븐(Data Driven) 경영 방식 도입
신규 가치 제안 창출	아마존	고객 주문 전에 예측 배송
	GE	산업 인터넷으로 스마트 공장 구현
	후지쯔	농업용 빅데이터 분석 솔루션 제공

© KOPO-Dept.
AI Engineering Limited

데이터 분석 활용 사례

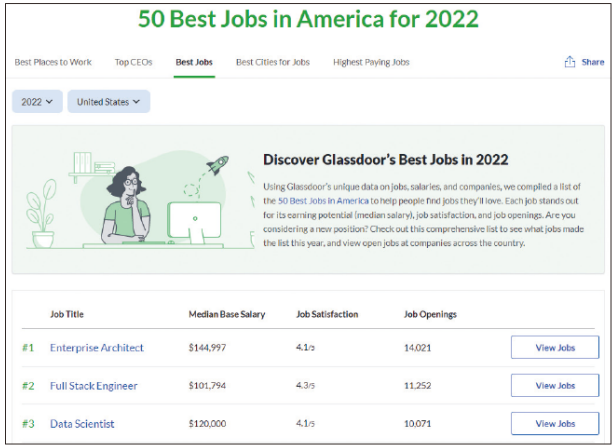
- 국내 활용사례
  - 신한카드, 엔씨소프트, 이마트, SK텔레콤 등 국내 기업들이 데이터 분석 기술을 선택이 아닌 필수적으로 활용

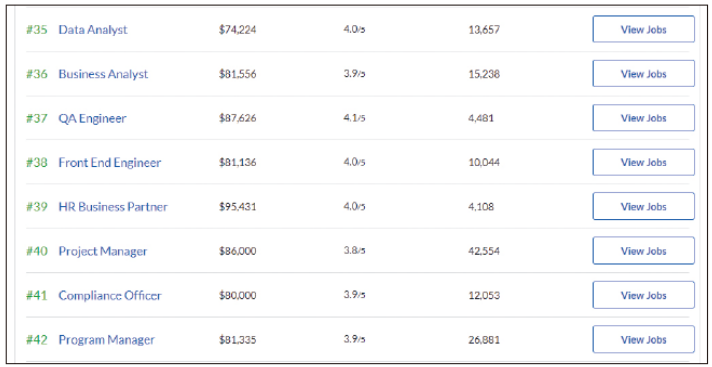
분류	국내 기업	활용 내용
고객 관계 관리 / 경험의 변화	신한카드	고객 라이프 스타일 맞춤형 상품
	KB국민카드	소비자 정보 기반 카드 이용 및 편의성 서비스
	엔씨소프트	사기 탐지(Fraud Detection) 알고리즘을 통한 불법 행위 모니터링
	옥션, G마켓, 11번가	고객 구매력에 따른 개인화된 상품 추천과 광고 배달
내부 프로세스 / 효율성 개선	삼성SDS	제조업 생산성 향상 지원 솔루션
	이마트	데이터 분석을 통해 의사결정을 내리는 데이터 드리븐(Data Driven) 경영 방식으로 생산한 자체생산(PB) 상품
신규 가치 제안 창출	SK텔레콤	상권 분석 및 타켓 마케팅 지원 서비스

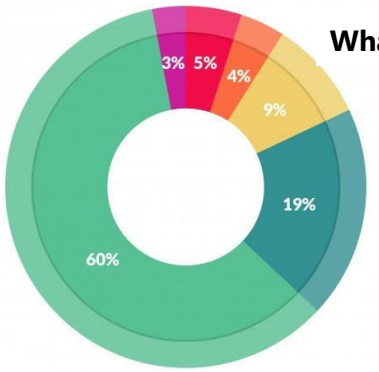
© KOPO-Dept.
AI Engineering Limited

데이터 분석 직업

- 데이터 과학자 및 분석가
  - 데이터 분석을 전문으로 하는 사람





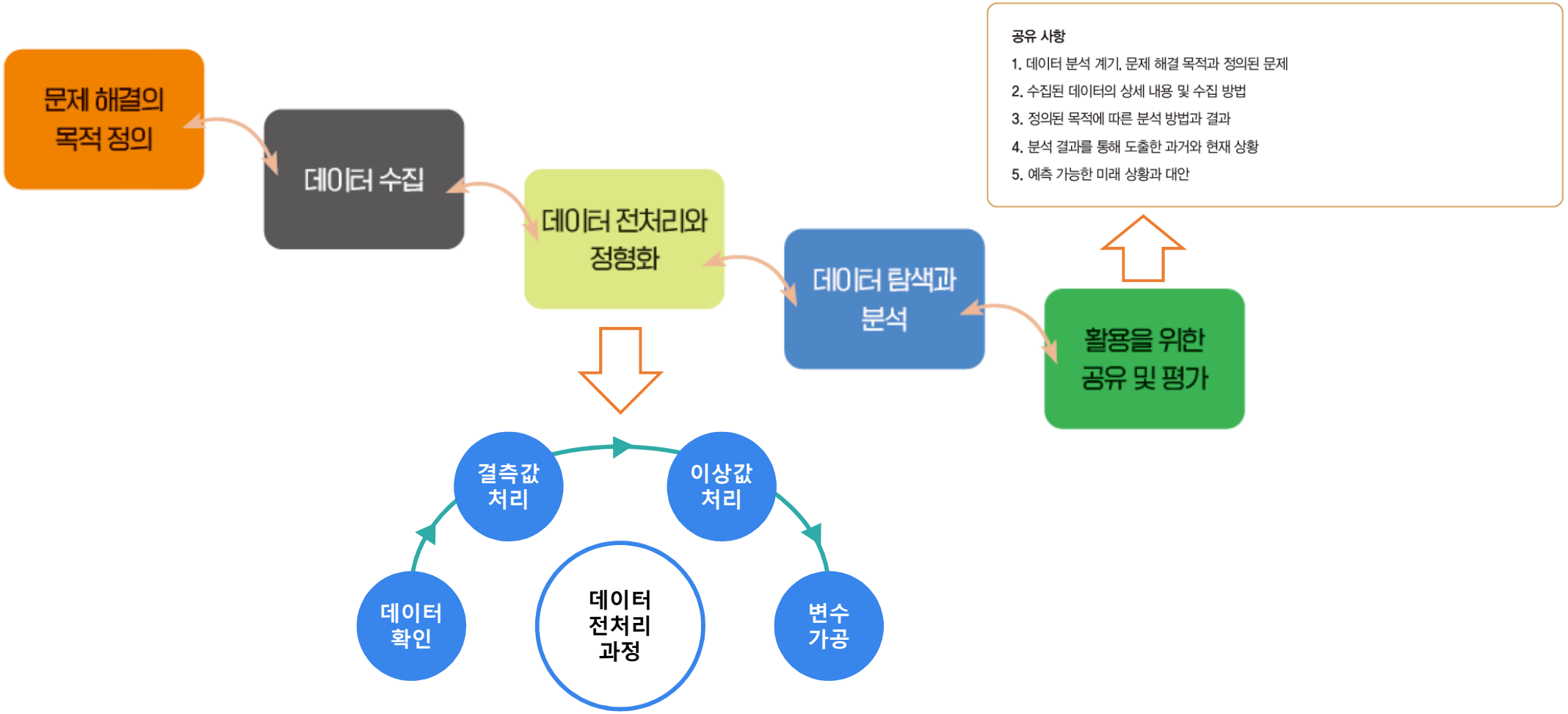


What data scientists spend the most time doing

- Building training sets : 3%
- Cleaning and organizing data : 60%
- Collecting data sets : 19%
- Mining data for patterns : 9%
- Refining algorithms : 4%
- Other : 5%

*출처: Big Data: The Next Frontier for Innovation, Competition, and Productivity, McKinsey Global Institute , 2011

데이터 분석 처리 과정



 © KOPO-Dept.
AI Engineering Limited

판다스 입문

- 데이터과학자가 판다스를 배우는 이유

빅데이터의 시대. 데이터 과학이라는 새로운 영역의 출현.

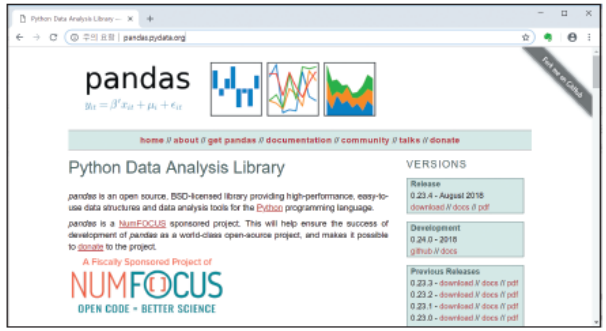
- 클라우드 컴퓨팅의 확산. 빅데이터 저장, 분석에 필요한 컴퓨팅 자원이 매우 저렴해짐.
- 컴퓨팅 파워의 대중화는 최적의 학습환경과 연구 인프라를 제공.

데이터과학은 데이터를 연구하는 분야이고, 데이터 자체가 가장 중요한 자원

- 데이터 분석 업무의 **80~90%는 데이터를 수집하고 정리하는 일**이 차지.
- 나머지 10~20%는 알고리즘을 선택하고, 모델링 결과를 분석하여 데이터로부터 유용한 정보(information)을 뽑아내는 분석 프로세스의 몫.
- 데이터과학자가 하는 가장 중요한 일이 **데이터를 수집하고 분석이 가능한 형태로 정리**하는 것.

판다스는 데이터를 수집하고 정리하는데 최적화된 도구.

- 가장 배우기 쉬운 프로그래밍 언어, 파이썬(Python) 기반.
- 오픈소스(open source)로 무료로 이용 가능.



[그림 1-1] 판다스 공식 홈페이지(http://pandas.pydata.org/)

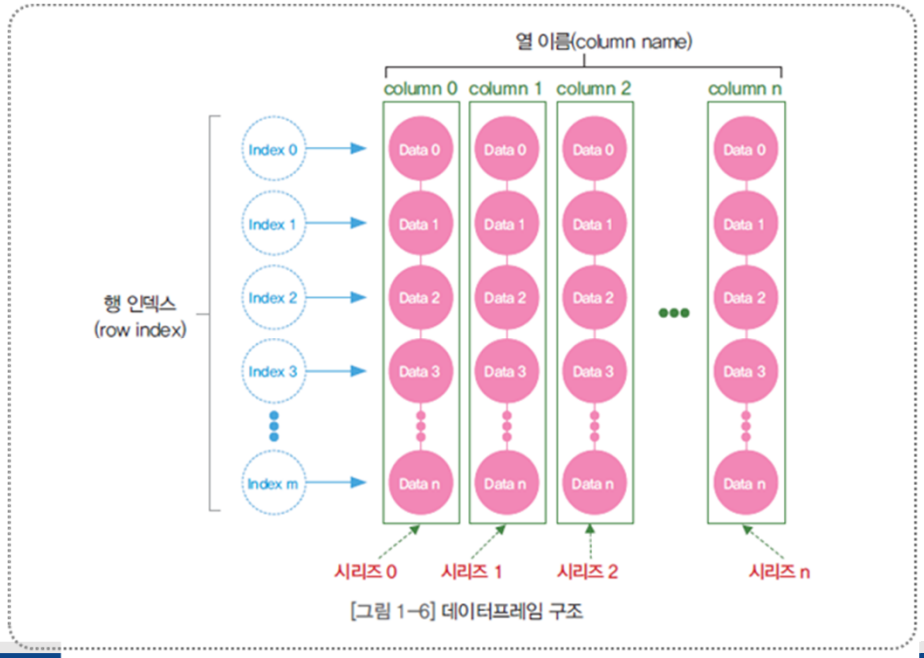
판다스 입문

- 판다스 자료구조

  - **시리즈(Series)와 데이터프레임(DataFrame)이라는 구조화된 데이터 형식**을 제공
  - 서로 다른 종류의 데이터를 한곳에 담는 컨테이너
  - 시리즈는 1차원 배열이고, 데이터프레임이 2차원 배열이라는 점에서 차이가 있음
  - 특히, **행과 열로 이루어진 2차원 구조의 데이터프레임은 데이터 분석 실무에서 자주 사용**



[그림 1-2] 시리즈 구조



[그림 1-6] 데이터프레임 구조

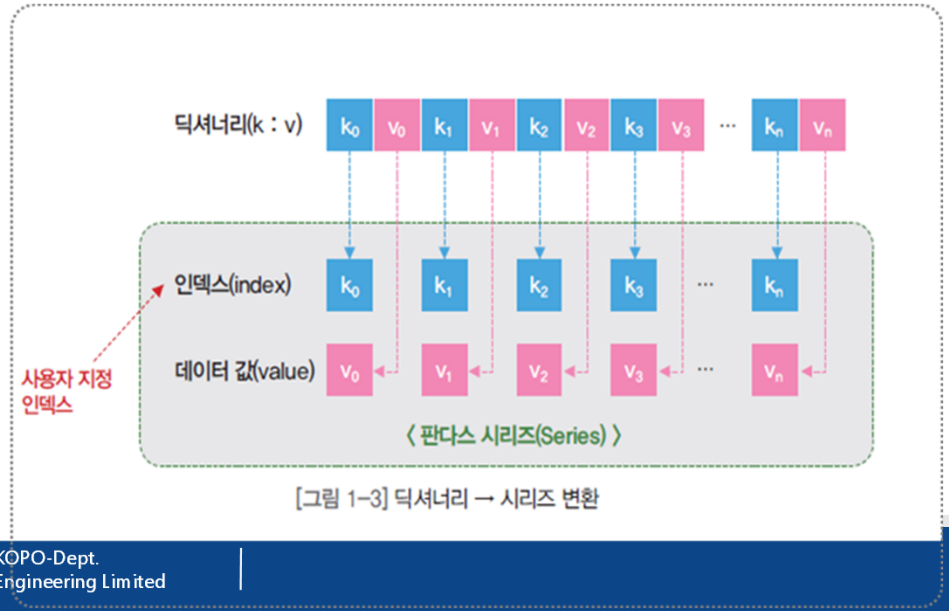
Daegu Campus of KOREA POLYTECHNIC

판다스 입문

- 판다스 자료구조 (시리즈)

  - 시리즈 만들기
    1) 딕셔너리와 시리즈의 구조가 비슷하기 때문에, **딕셔너리를 시리즈로 변환하는 방법을 많이 사용**
    2) 판다스 내장 함수인 **Series()를 이용**하고, 딕셔너리를 함수의 매개변수(인자)로 전달
    3) 딕셔너리의 키(k)는 시리즈의 인덱스에 대응하고, 딕셔너리의 각 키에 매칭되는 값(v)이 시리즈의 데이터 값(원소)로 변환.

딕셔너리 → 시리즈 변환: `pandas.Series( 딕셔너리 )`



[그림 1-3] 딕셔너리 → 시리즈 변환

```python
import pandas as pd

# key:value 구조를 갖는 딕셔너리를 만들고, 변수 dict_data에 저장
dict_data = {'a': 1, 'b': 2, 'c': 3}

# 판다스 Series() 함수로 딕셔너리(dict_data)를 시리즈로 변환. 변수 sr에 저장
sr = pd.Series(dict_data)

# 변수 sr의 자료형 출력
print(type(sr))

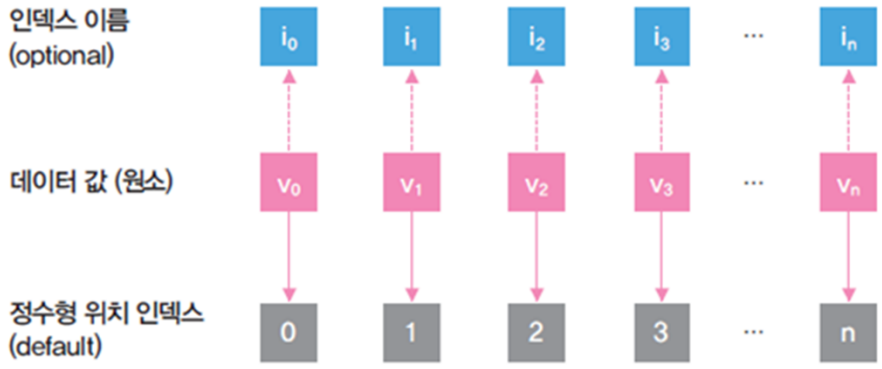
# 변수 sr에 저장되어 있는 시리즈 객체를 출력
print(sr)
```


© KOPO-Dept.
AI Engineering Limited

판다스 입문

- 판다스 자료구조 (시리즈)

  - 인덱스 구조
    1) 인덱스는 자기와 짝을 이루는 원소(데이터 값)의 순서와 주소를 저장
    2) 인덱스를 잘 활용하면 데이터 값의 탐색, 정렬, 선택, 결합 등 데이터 조작
    3) 인덱스의 종류 (2가지)
       - 정수형 위치 인덱스(integer position): Default
       - 인덱스 이름(index name) 또는 인덱스 라벨(index label)



[그림 1-4] 시리즈 인덱스의 유형

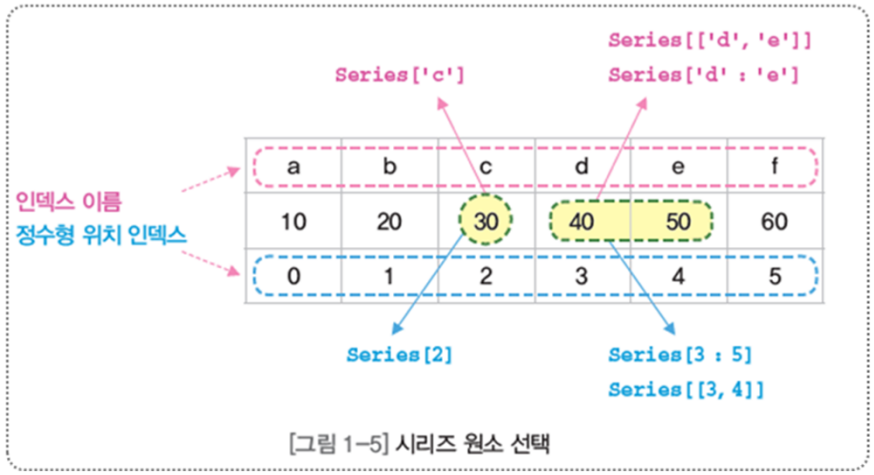
```
# 리스트를 시리즈로 변환하여 변수 sr에 저장
list_data = ['2019-01-02', 3.14, 'ABC', 100, True]
sr = pd.Series(list_data)
print(sr)

# 인덱스 배열은 변수 idx에 저장. 데이터 값 배열은 변수 val에 저장
idx = sr.index # RangeIndex 객체로 반환
val = sr.values # list_data 리스트 원소의 순서를 유지한 상태로 반환
print(idx)
print(val)
```

판다스 입문

- 판다스 자료구조 (시리즈)

  - 원소 선택
    1) 인덱스를 이용하여 시리즈의 원소 선택
    2) 하나의 원소를 선택하거나, 여러 원소를 한꺼번에 선택 가능 e.g.) sr[0], sr[[1, 2]]
    3) 인덱스의 범위를 지정하여 여러 개의 원소 선택 가능 (슬라이싱, e.g.) sr[0:3]
    4) 인덱스의 유형에 따라 사용법이 조금 다름
       - 정수형 인덱스: [ ] 안에 숫자 입력 (Indexing)
       - 인덱스 이름(라벨): [ ] 안에 이름을 입력



[그림 1-5] 시리즈 원소 선택

© KOPO-Dept.
AI Engineering Limited

판다스 입문

- 판다스 자료구조 (시리즈)

  - 원소 선택
    1) 인덱스를 이용하여 시리즈의 원소 선택
    2) 하나의 원소를 선택하거나, 여러 원소를 한꺼번에 선택 가능 e.g.) sr[0], sr[[1, 2]]
    3) 인덱스의 범위를 지정하여 여러 개의 원소 선택 가능 (슬라이싱, e.g.) sr[0:3]

```
# 투플을 시리즈로 변환(index 옵션에 인덱스 이름을 지정)
tup_data = ('홍길동', '19xx-xx-xx', '남', True)
sr = pd.Series(tup_data, index=['이름', '생년월일', '성별', '학생여부'])
print(sr)

# 원소를 1개 선택
print(sr[0])      # sr의 1 번째 원소를 선택 (정수형 위치 인덱스를 활용)
print(sr['이름']) # '이름' 라벨을 가진 원소를 선택 (인덱스 이름을 활용)

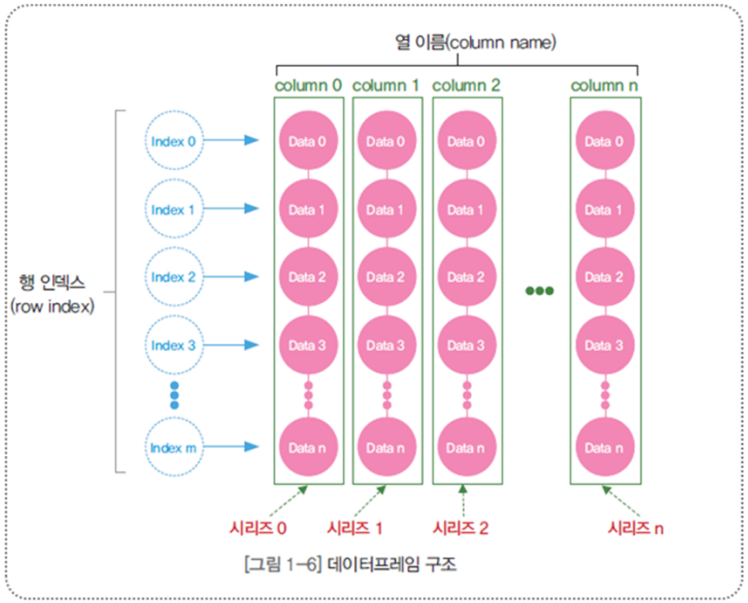
# 여러 개의 원소를 선택 (인덱스 리스트 활용)
print(sr[[1, 3]])
print(sr[['생년월일', '학생여부']])
print(sr[1:4]) # 생년월일~학생여부
print(sr['생년월일': '학생여부'])
```


© KOPO-Dept.
AI Engineering Limited

판다스 입문

- 판다스 자료구조 (데이터프레임)

  - 개요
    1) 2차원 배열 (R의 데이터프레임에서 유래)
    2) 데이터프레임의 열은 시리즈 객체. 시리즈를 열벡터(vector)라고 하면, 데이터프레임은 여러 개의 열벡터들이 같은 행 인덱스를 기준으로 줄지어 결합된 **행렬(matrix)**
    3) 데이터프레임은 행과 열을 나타내기 위해 두 가지 종류의 주소를 사용.
       - 행 인덱스(row index)와 열 이름(column name 또는 column label)으로 구분



[그림 1-6] 데이터프레임 구조

- 데이터프레임의 각 열은 공통의 속성을 갖는 일련의 데이터를 나타냄.
- 각 행은 개별 관측대상에 대한 다양한 속성 데이터들의 모음인 레코드(record).

열

종목 코드	회사 이름	액면가	총 주식수
005930	삼성전자	100원	5,970백만 주
017670	SK텔레콤	500원	81백만 주
005380	현대자동차	5000원	214백만 주

행

[표 1-1] 주식 종목 리스트

Daegu Campus of KOREA POLYTECHNIC

판다스 입문

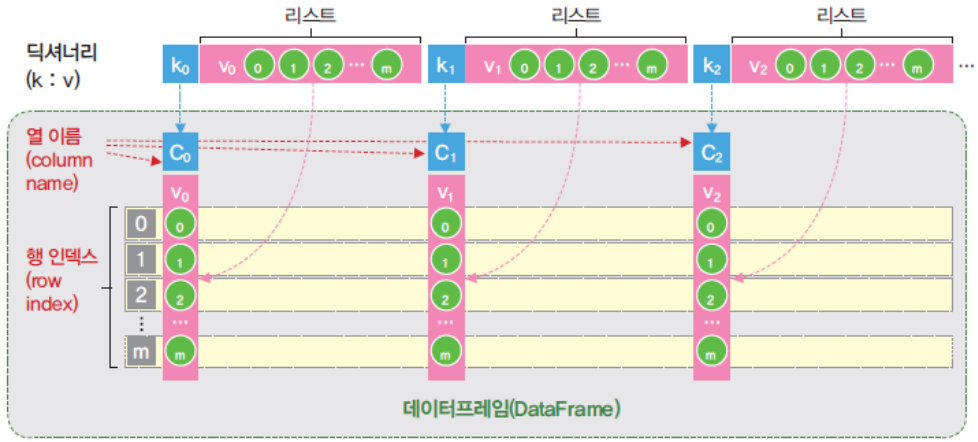
- 판다스 자료구조 (데이터프레임)

  - 데이터프레임 만들기

    1) 같은 길이(원소의 개수가 동일한)의 배열 여러 개가 필요. 데이터프레임은 여러 개의 시리즈(열, column)를 모아 놓은 집합
    2) 판다스 DataFrame() 함수를 사용. 여러 개의 리스트를 원소로 갖는 딕셔너리를 함수에 전달하는 방식을 주로 활용

    딕셔너리 → 데이터프레임 변환: `pandas.DataFrame( 딕셔너리 객체 )`

    3) 딕셔너리의 값(v)에 해당하는 각 리스트가 시리즈로 변환되어 데이터프레임의 각 열이 됨
    4) 딕셔너리의 키(k)는 각 시리즈의 이름으로 변환되어, 최종적으로 데이터프레임의 열 이름이 됨



[그림 1-7] 딕셔너리 → 데이터프레임 변환

```
import pandas as pd

# 열이름을 key로 하고, 리스트를 value로 갖는 딕셔너리 정의(2차원 배열)
dict_data = {'c0':[1,2,3], 'c1':[4,5,6], 'c2':[7,8,9],
             'c3':[10,11,12], 'c4':[13,14,15]}

# 판다스 DataFrame() 함수로 딕셔너리를 데이터프레임으로 변환. 변수 df에 저장.
df = pd.DataFrame(dict_data)

# df의 자료형 출력
print(type(df))
# 변수 df에 저장되어 있는 데이터프레임 객체를 출력
print(df)
```

판다스 입문

- 판다스 자료구조 (데이터프레임)

  - 행 인덱스/열 이름 설정: 데이터 프레임의 행 인덱스와 열 이름을 사용자가 지정 가능

    1) 데이터프레임을 만들때 설정

```
행 인덱스/열 이름 설정: pandas.DataFrame( 2차원 배열,
                                        index=행 인덱스 배열,
                                        columns=열 이름 배열 )
```

    2) 속성을 지정하여 변경: **df.index, df.column**으로 행 인덱스/열 이름 배열에 접근 가능(수정 가능)

```
• 행 인덱스 변경: DataFrame 객체.index = 새로운 행 인덱스 배열
• 열 이름 변경: DataFrame 객체.columns = 새로운 열 이름 배열
```

    3) rename 함수 사용: 새로운 데이터프레임 객체를 리턴, **원본 객체를 변경하려면, inplace=True 옵션을 사용**

```
• 행 인덱스 변경: DataFrame 객체.rename(index={기존 인덱스:새 인덱스, … })
• 열 이름 변경: DataFrame 객체.rename(columns={기존 이름:새 이름, … })
```


© KOPO-Dept.
AI Engineering Limited

판다스 입문

- 판다스 자료구조 (데이터프레임)
  - 행 인덱스/열 이름 설정

```
df = pd.DataFrame([ [23, '남', '서울'],
                    [24, '여', '대구'],
                    [22, '여', '대전'] ],
                    index=['홍길동', '신수민', '신수아'],
                    columns=['나이', '성별', '지역'])

# 행 인덱스, 열 이름 확인하기
print(df)          #데이터프레임

# 행 인덱스, 열 이름 변경하기 (내장 함수 사용)
df.index = ['학생1', '학생2', '학생3']
df.columns = ['연령', '남녀', '주소지']
print(df)

df.rename(index={'학생1': '홍길동', '학생2':'신수민', '학생3':'신수아'}, inplace=True)
```

판다스 입문

- 판다스 자료구조 (데이터프레임)

  - 행/열 삭제
    1) drop() 메소드에 행을 삭제할 때는 축(axis) 옵션으로 axis=0을 입력 (Default: axis = 0)
    2) axis = 1 입력 시: 열 삭제
    3) 여러 개의 행 또는 열을 삭제하려면, 리스트 형태로 입력
    4) drop() 메소드는 기존 객체를 변경하지 않고, 새로운 객체를 반환, **원본 객체를 변경하려면, inplace=True 옵션을 사용**

- **행 삭제:** `DataFrame 객체.drop(행 인덱스 또는 배열, axis=0)`
- **열 삭제:** `DataFrame 객체.drop(열 이름 또는 배열, axis=1)`

```
# DataFrame() 함수로 딕셔너리 데이터를 데이터프레임으로 변환 후 변수 df에 저장
exam_data = {'인공지능기초' : [ 90, 80, 70], '자료구조' : [ 98, 89, 95],
             '자바' : [ 85, 95, 100], '기계학습프로그래밍' : [ 100, 90, 90]}

df = pd.DataFrame(exam_data, index=['학생1', '학생2', '학생3'])
print(df)
# df의 1개 행(row)을 삭제 후 df2에 저장
df2 = df.drop('학생2') # axis=0 default
print(df2)
```

판다스 입문

- 판다스 자료구조 (데이터프레임)

  - 행 선택

    1) loc, iloc 인덱서를 사용

       - loc: 인덱스 이름을 기준으로 행을 선택할 때
       - iloc: 정수형 위치 인덱스를 사용할 때

  - 열 선택

    열 1개 선택(시리즈 생성): `DataFrame 객체["열 이름"]` 또는 `DataFrame 객체.열 이름`

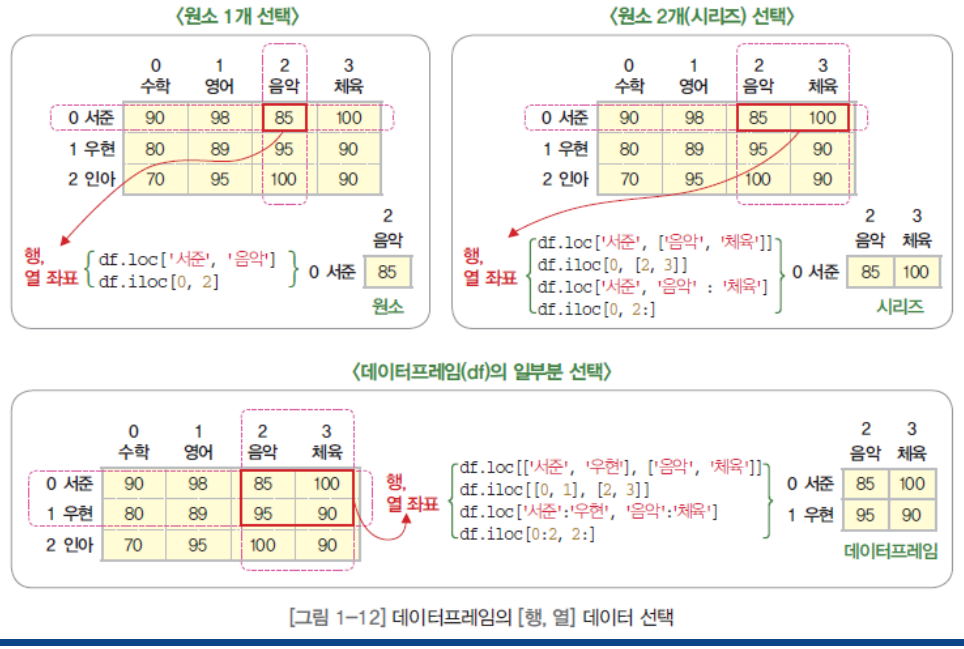
    열 n개 선택(데이터프레임 생성): `DataFrame 객체[ [ 열1, 열2 , … , 열n ] ]`

  - 원소 선택

    - **인덱스 이름: DataFrame 객체.loc[행 인덱스, 열 이름]**
    - **정수 위치 인덱스: DataFrame 객체.iloc[행번호, 열번호]**

구분	loc	iloc
탐색 대상	인덱스 이름(index label)	정수형 위치 인덱스(integer position)
범위 지정	가능(범위의 끝 포함) 예) `['a':'c']` → `'a',' b', 'c'`	가능(범위의 끝 제외) 예) `[3:7]` → `3, 4, 5, 6` (* 7 제외)

[표 1-2] loc과 iloc



[그림 1-12] 데이터프레임의 [행, 열] 데이터 선택

 © KOPO-Dept.
AI Engineering Limited

판다스 입문

- 판다스 자료구조 (데이터프레임)

  - 행 선택, 열 선택, 원소 선택

```
exam_data = {'인공지능기초' : [ 90, 80, 70], '자료구조' : [ 98, 89, 95],
             '자바' : [ 85, 95, 100], '기계학습프로그래밍' : [ 100, 90, 90]}

df = pd.DataFrame(exam_data, index=['학생1', '학생2', '학생3'])
print(df)      # 데이터프레임 출력

# 행 인덱스를 사용하여 행 1개를 선택
label1 = df.loc['학생1']   # loc 인덱서 활용
position1 = df.iloc[0]     # iloc 인덱서 활용
print("====label1")
print(label1, type(label1))
print("====position1")
print(position1, type(position1))

# 행 인덱스를 사용하여 2개 이상의 행 선택
label2 = df.loc[['학생1', '학생3']]
position2 = df.iloc[[0, 2]]
print(label2, type(label2))
print(position2)

# 행 인덱스의 범위를 지정하여 행 선택
label3 = df.loc['학생1':'힉셍3']
position3 = df.iloc[0:3]
print(label3)
print(position3)
```

판다스 입문

- 판다스 자료구조 (데이터프레임)

  - 열 (Column) 추가
    1) 추가하려는 열 이름과 데이터 값을 입력 (➔ 마지막 열에 덧붙이듯 새로운 열 추가)
    2) 값을 하나만 지정하면 모든 행에 동일한 값이 입력됨

```
열 추가: DataFrame 객체[ '추가하려는 열 이름' ] = 데이터 값
```



[그림 1-13] 데이터프레임의 열 추가

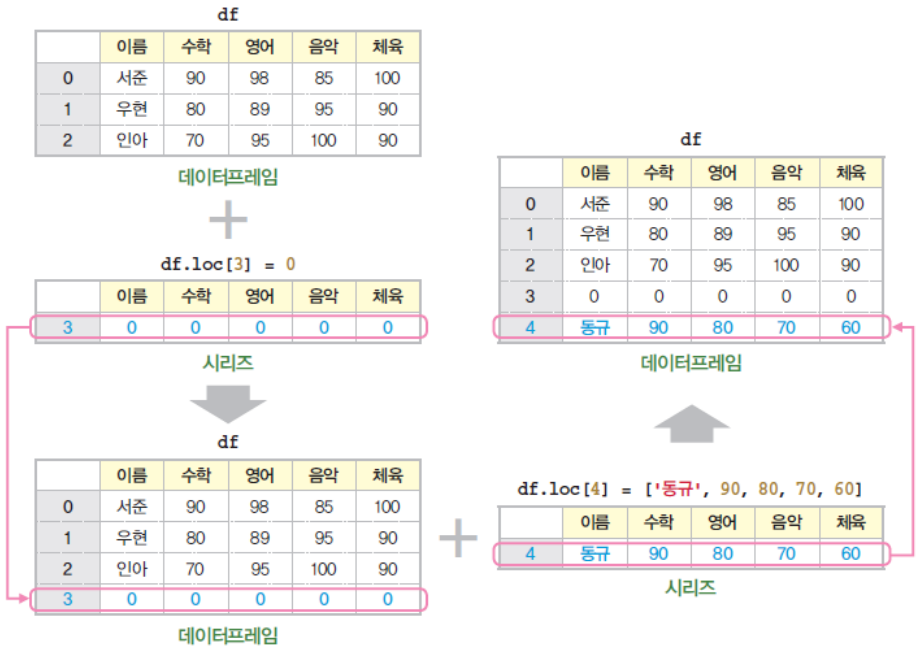
판다스 입문

- 판다스 자료구조 (데이터프레임)

  - 행 (row) 추가

    1) 행 인덱스와 데이터 값을 **"loc 인덱서를 사용"**하여 입력

행 추가: DataFrame.loc['새로운 행 이름'] = 데이터 값 (또는 배열)



[그림 1-14] 데이터프레임의 행 추가

©
A

판다스 입문

- 판다스 자료구조 (데이터프레임)

  - 원소 값 변경
    1) 원소를 선택하고 새로운 데이터 값을 지정

```
원소 값 변경: DataFrame 객체의 일부분 또는 원소를 선택 = 새로운 값
```

```
df.iloc[0, 3] = 80  # 원소 한 개 수정
df.iloc[0, [2, 3]] = 100, 80 # 원소 여러 개(두 개)를 한 번에 수정
```

  - 행, 열의 위치 바꾸기
    1) 데이터프레임의 행과 열을 맞바꾸는 방법
    2) 전치 결과는 새로운 객체를 반환하므로, 기존 객체를 변경하기 위해서는 df = df.transpose() 해야 함

```
행, 열 바꾸기: DataFrame 객체.transpose() 또는 DataFrame 객체.T
```


© KOPO-Dept.
AI Engineering Limited

판다스 입문

- 판다스 자료구조 (데이터프레임)

  - 행, 열 추가, 원소 값 변경

```
exam_data = {'AI기초' : [ 90, 80, 70],
             '자료구조' : [ 98, 89, 95],
             '자바' : [ 85, 95, 100],
             '데이터전처리' : [ 100, 90, 90]}
df = pd.DataFrame(exam_data, index=['홍길동', '신수민', '신수아'])
print(df)
# 새로운 컬럼을 추가 - 마지막 컬럼에 추가 됨. (값을 하나 지정하면 동일한 값으로 모든 Row에 적용됨)
# df['웹프로그래밍'] = 80
df['웹프로그래밍'] = [80, 100, 90]
print(df)

# 새로운 행(row)을 추가 - 같은 원소 값을 입력
# index가 '학생4'인 행을 추가하고 모든 컬럼 값은 0
df.loc['학생4'] = 0
print(df)

# 새로운 행(row)을 추가 - 원소 값 여러 개의 배열 입력
# index가 '학생5'인 행을 추가 (컬럼 값은 지정)
df.loc['학생5'] = [90, 80, 70, 60, 80]
print(df)
```

Thank you

Q&A

www.kopo.ac.kr
jsshin7@kopo.ac.kr

 © KOPO-Dept.
AI Engineering Limited